# Project EUREKA

DGIPR GOVERNMENT OF MAHARASHTRA · COMMAND CENTER CORE ENGINE

BACKEND & PIPELINE SPECIFICATION

Version 1.0 · Production Architecture

## 1. Architecture Overview

**Project EUREKA** serves as the high-throughput media intelligence, factual verification, and composite district evaluation engine for the Directorate General of Information and Public Relations (DGIPR), Government of Maharashtra.

The backend couples an asynchronous **FastAPI ASGI service** with distributed message queues (Celery/Redis), relational and graph persistence (PostgreSQL 16 & Neo4j 5.x), and machine-learned inference (Google Gemma/Gemini & MuRIL Indic NLP).

> **Runtime Operational Status:** The Project EUREKA ASGI server runs locally on `http://127.0.0.1:8000` and is reversely proxied through the Next.js frontend on port `3001` at `/eureka/` and `/api/v1/`.

## 2. Core Backend Server & Entrypoints

| File Path | Component | Description & Key Responsibilities |
|---|---|---|
| `backend/main.py` | ASGI ENTRY | Top-level entrypoint for Uvicorn/Vercel. Injects `backend/src` into `sys.path` and exports the production app instance. |
| `backend/src/eureka/api/main.py` | FASTAPI CORE | Application bootstrap, lifecycle manager (Neo4j constraint initialization and shutdown), CORS middleware, and router registration across all 12 operational modules. |
| `backend/src/eureka/core/config.py` | SETTINGS | Pydantic Settings module handling `DATABASE_URL`, PostgreSQL connection pooling, Neo4j URIs, Redis brokers, JWT secrets (HS256), WebAuthn RP settings, and Gemini API keys. |
| `backend/src/eureka/core/database.py` | PERSISTENCE | SQLAlchemy async engine setup using `asyncpg`, session factory (`get_async_db`), connection pooling (pool size 20, max overflow 10), and database telemetry health check. |
| `backend/src/eureka/models/entities.py` | DATA SCHEMA | Declarative ORM models for Canonical Events, Fact-Check Tickets, Audit Logs, Pillar Score Snapshots, and Outbox event projections. |

## 3. Backend Data Pipelines

### 1. Hermes 90-Minute Crisis SLA & Rebuttal Pipeline

SLA DISPATCHER

`backend/src/eureka/hermes/dm_pipeline_manager.py`

Orchestrates the statutory **90-minute crisis lifecycle** across all 36 districts of Maharashtra:

- **Incident Ingestion:** Ingests urgent accident reports and misinformation alerts with timestamp verification.
- **AI Rebuttal Drafting:** Generates bilingual (English/Marathi) evidence-backed factual counter-narratives via Google AI Studio.
- **State Machine & RBAC:** Enforces role-based transitions: *Detected* → *Telemetry Verified* → *Rebuttal Authorized* → *Wire Broadcast*.
- **Executive Dossiers 1–4:** Automates real-time situational reporting for CMO, DCM, and District Collectors.

### 2. SAVI Dynamic DAG Ingestion & Clustering Pipeline

INGESTION ROUTER

`src/savi/pipeline/dag_router.py`

Phase 2 Dynamic DAG controller routing multi-platform ingestion envelopes:

- **Micro-Event Pipeline:** Parallelized execution for isolated local alerts.
- **Macro-Cascade Temporal Buffer:** Aggregates cross-platform items sharing entity overlap within temporal window ($\Delta t \leq 15$ min).
- **Anomaly Quarantine:** Passes grouped envelopes through the Isolation Rainforest module to filter synthetic or coordinated bot surges.

### 3. 100-Point Composite Performance Index (CPI) Pipeline

SCORING ENGINE

`backend/src/eureka/scoring/scoring_engine.py & tasks/scoring_tasks.py`

Calculates statutory administrative performance across all 36 Maharashtra districts based on 4 pillars:

- **Pillar 1: Media Release Volume (25 pts):** Output cadence across 5 regional languages.
- **Pillar 2: Response TAT & Citizen Inquiries (25 pts):** Grievance resolution velocity within SLA.
- **Pillar 3: Multi-Channel Public Sentiment (25 pts):** Composite sentiment computed across Print, TV, and Social Radar.
- **Pillar 4: Crisis SLA Resolution Rate (25 pts):** Adherence to statutory <4h incident resolution.
- **Celery Task Offload:** Scheduled periodic re-indexing via Redis broker queue.

### 4. Change Data Capture (CDC) & Outbox Pipeline

EVENT STREAMING

`src/savi/persistence/cdc_worker.py & backend/src/eureka/conclusion/outbox_worker.py`

Guarantees transactional event delivery from the relational database:

- **Outbox Polling:** Continuously monitors uncommitted events in batches of 100 with exponential backoff.
- **Graph Synchronizer:** Reflects verified events into Neo4j graph nodes and Louvain community clusters.
- **Zero-Loss Guarantee:** Implements idempotent consumer tokens to prevent duplicate telemetry alerts.

### 5. Regional Newspaper Web Scraping Pipeline

SCRAPING ENGINE

`NEWS_API.py`

Multithreaded web scraper indexing major Marathi and English dailies (Lokmat, Maharashtra Times, Sakaal, Pudhari, Navakal, Indian Express, Times of India):

- Extracts articles, OCR summaries, publication timestamps, and author bylines.
- Feeds validated article links directly into the Live News Queue.

## 4. API Routers & Endpoints Directory

| Router Module | Endpoint Path | Method | Description |
|---|---|---|---|
| `routers/health.py` | `/api/v1/health` | GET | System health & telemetry check (Postgres, Neo4j, Copilot). |
| `routers/scoring.py` | `/api/v1/scoring/leaderboard` | GET | Ranked 36-district performance scores across all 4 pillars. |
| `routers/scoring.py` | `/api/v1/scoring/district/{id}` | GET | Detailed pillar decomposition for a single district. |
| `routers/hermes_router.py` | `/api/v1/hermes/rebuttal/action` | POST | Authorizes, issues advisories, or requests edits on crisis alerts. |
| `routers/conclusions.py` | `/api/v1/conclusions/resolve` | POST | Records cross-desk consensus and concludes verified events. |
| `routers/graph.py` | `/api/v1/graph/narrative-clusters` | GET | Queries Neo4j narrative knowledge graph and triplet links. |
| `api/auth.py` | `/api/v1/auth/token` | POST | Generates 12-hour signed JWT bearer tokens with district RBAC. |
| `routers/webauthn_router.py` | `/api/v1/auth/webauthn/verify` | POST | FIDO2 biometric verification (zero-server-biometric storage). |

## 5. Live Telemetry & Interactive Frontend Monitoring

The backend pipeline flow is monitored directly through the Next.js Command Center interface:

- **Interactive Pipeline Dashboard:** `http://localhost:3001/command-center/pipeline-flow`
- **Animated Stage Mesh:** `src/components/commandCenter/AnimatedPipelineDiagram.tsx`
- **Pipeline Stage Inspector:** `src/components/commandCenter/PipelineFlowView.tsx`
- **Interactive API Swagger Docs:** `http://127.0.0.1:8000/docs`